HealthAI → Piattaforma B2B Cardio-Nefro-Metabolica

Documento CTO / Prompt esecutivo per Claude Code / Claude CoWork

Obiettivo: rifocalizzazione verticale, hardening sicurezza, multi-paziente, report PDF, alert clinici, conformità GDPR e readiness pubblicazione

Basato sull'analisi dei file allegati dell'app attuale

Asse	Sintesi
Stato attuale	App ibrida patient-facing con score, chatbot, nutrizione, attività fisica, dashboard e Supabase nel frontend
Stato target	Piattaforma B2B per professionisti sanitari focalizzata su valutazione e follow-up del rischio cardio-nefro-metabolico
Vincolo chiave	NON modificare le logiche di calcolo degli score già testate; fare refactor architetturale e integrazione intorno ad esse

1. Mandato di implementazione

Questo documento deve essere usato come specifica tecnica e prompt esecutivo per Claude Code / Claude CoWork. Il task non è una semplice refactor estetica: Claude deve trasformare l'attuale web app in una piattaforma B2B focalizzata sul rischio cardio-nefro-metabolico, mantenendo intatte le logiche di calcolo degli score già validate, eliminando le componenti fuori verticale, mettendo in sicurezza l'architettura e preparando il prodotto alla pubblicazione.

- **Preservare:** SCORE2, SCORE2-Diabetes, ADA, FLI, FRAIL e le altre formule esistenti già testate; non cambiare formule, coefficienti, soglie o output clinici senza motivazione esplicita e test di equivalenza.
- **Mantenere:** valutazione alimentare / qualità della dieta e monitoraggio aderenza, ma non creare piani alimentari prescrittivi.
- **Mantenere con riduzione:** valutazione dell'attività fisica come dato clinico e di rischio, ma eliminare generazione di programmi di allenamento.
- **Eliminare o declassare:** chatbot generalista, modalità sintomi, programmi di allenamento, piani alimentari completi, feature consumer poco professionali.
- **Aggiungere:** profili multipli di pazienti, storico valutazioni, report PDF professionale, timeline follow-up, alert intelligenti per professionista e paziente, audit trail, hardening sicurezza, consenso e governance GDPR.

2. Evidenze ricavate dalla codebase attuale

Dall'analisi dei file allegati emergono pattern architetturali che spiegano perché il prodotto oggi è poco focalizzato e non ancora pronto alla pubblicazione.

- **Segreti e configurazione nel frontend:** URL Supabase e anon key sono inseriti in HTML/JS pubblici, con duplicazioni in più file e ambienti diversi.
- **Identificazione record tramite email:** diverse query leggono e aggiornano anagrafica_utenti con .eq("email", email) invece di usare auth.uid() / user_id.
- **Logica score frammentata:** molti score vengono precompilati in iframe separati e il risultato viene recuperato via postMessage, con listener sparsi e logica di persistenza replicata.
- **Prodotto troppo largo:** la dashboard contiene nutrizione, screening, stile di vita, training plan, raccomandazioni generative, e il chatbot apre casi d'uso non coerenti con il verticale B2B.
- **GDPR incompleto:** esiste un modale consenso, ma non si vede un modello robusto di versioning del consenso, data retention, minimizzazione, data subject rights, audit, purpose limitation e segregazione per tenant.
- **Multi-paziente assente:** il modello attuale è centrato sull'utente autenticato che salva il proprio profilo, non sul professionista che gestisce una lista di pazienti.

3. Decisione di prodotto: cosa deve diventare l'app

La piattaforma target non è più una wellness app generica. Deve diventare un software B2B clinico leggero, orientato a professionisti sanitari, per screening, stratificazione del rischio, monitoraggio nel tempo e supporto al follow-up cardio-nefro-metabolico.

Area	Tenere	Rimuovere / Limitare	Target finale
Score clinici	SCORE2, SCORE2-Diabetes, ADA, FLI, sindrome metabolica, BMI, parametri base	Nessuna modifica alle formule	Motore unico deterministico
Nutrizione	PREDIMED/aderenza, qualità dieta, suggerimenti AI generici e non prescrittivi	Piano alimentare completo	Modulo monitoraggio aderenza
Attività fisica	Valutazione quantitativa e qualitativa come fattore di rischio	Programma di allenamento personalizzato	Campo clinico, non coach fitness
Chat/AI	Solo sintesi/report/educazione controllata e opzionale	Chatbot generalista, modalità sintomi aperta	Assistente professionale limitato e governato
Utenti	Professionisti + pazienti	Single-user consumer model	B2B multi-tenant con ruoli
Output	PDF clinico, alert, trend, follow-up	Contenuti dispersivi	Valore operativo per medico e paziente

4. Vincoli non negoziabili per Claude

1. Non alterare le formule degli score esistenti già testati; se serve rifattorizzare, estrarre la logica in moduli puri e aggiungere test snapshot/equivalence.
2. Non introdurre nuove dipendenze non necessarie nel frontend. Ogni nuova dipendenza deve essere giustificata.
3. Non lasciare chiavi, URL sensibili, policy version o configurazioni modificabili hardcodate nel frontend oltre al minimo pubblico strettamente necessario.
4. Non usare email come foreign key applicativa o filtro principale dei record clinici.
5. Non usare postMessage("*") senza controllo dell'origine e del payload.
6. Non salvare immagini/avatar sanitari in base64 direttamente nella tabella profilo se ciò gonfia il record o crea rischio; usare object storage con policy dedicate.
7. Non mantenere feature di allenamento o piano alimentare completo.
8. Non generare consigli clinici ad alto rischio con LLM senza regole deterministiche, disclaimers, consenso e tracciabilità.

5. Architettura tecnica target

- **Frontend web app:** interfaccia professionista e portale paziente, con componenti separati, router chiaro e stato per sessione/tenant.

- **Backend/API layer:** endpoint server-side per report PDF, orchestrazione alert, consenso, audit log, invio notifiche, eventuale generazione AI controllata.
- **Clinical domain layer:** motore deterministico separato in moduli puri: score engine, risk aggregation, screening engine, follow-up engine, diet adherence engine.
- **Persistence layer:** schema relazionale con tenants, professionals, patients, assessments, score_results, alerts, consent_records, audit_events.
- **Security layer:** Supabase Auth, RLS, policy per tenant e ruolo, secret management, CSP, CSRF strategy dove applicabile, rate limits, input validation, logging.
- **Compliance layer:** consensi versionati, finalità di trattamento, data minimization, retention, export/cancellation workflow, privacy by design.

6. Struttura cartelle consigliata

```
/app
  /frontend
    /public
    /src
      /app
        /auth
        /dashboard
        /patients
        /patient-detail
        /assessments
        /reports
        /settings
      /components
        /ui
        /charts
        /forms
        /scores
        /alerts
      /lib
        /api-client
        /auth
        /validation
        /formatters
      /styles
  /backend
    /src
      /api
        /auth
        /patients
        /assessments
        /reports
        /alerts
        /consents
        /webhooks
      /domain
        /clinical
          score-engine
          risk-aggregation
          screening-engine
          followup-engine
          nutrition-engine
        /security
        /audit
```

```
        /notifications
      /repositories
      /services
      /templates
        /pdf
      /config
      /middleware
      /utils
  /shared
    /types
    /constants
    /schemas
  /tests
    /unit
    /integration
    /e2e
    /fixtures
  /docs
    architecture.md
    api-spec.md
    threat-model.md
    gdpr-map.md
  /supabase
    /migrations
    /policies
    /seed
```

7. Schema database target

Lo schema deve essere ridisegnato per il modello B2B. Le tabelle principali devono distinguere tenant, professionista, paziente, assessment e risultati. I dati sanitari devono essere minimizzati e organizzati per snapshot temporali.

Tabella	Scopo	Colonne chiave
tenants	Organizzazione/cliente B2B	id, name, plan, status, created_at
users	Profilo utente applicativo	id(auth uid), tenant_id, role, full_name, email, status
professionals	Dati professionista	user_id, specialty, license_number, clinic_name
patients	Anagrafica paziente	id, tenant_id, created_by, external_code, display_name, sex, birth_year/date, contact prefs, consent status
patient_clinical_profiles	Dati stabili/di base	patient_id, smoking_status, diagnoses, medications, allergies, history
assessments	Snapshot visita/valutazione	id, patient_id, assessed_by, assessment_date, status, notes
assessment_measurements	Parametri quantitativi	assessment_id, bmi, waist_cm, sbp, dbp, total_chol, hdl, ldl, tg, glucose, hba1c, egfr, ggt, ast, alt, platelets
score_results	Output score deterministici	assessment_id, score_code,

Tabella	Scopo	Colonne chiave
		value_numeric, category, raw_payload_json, version
risk_profiles	Indice integrato e bucket	assessment_id, composite_risk_level, summary_json, action_flags_json
nutrition_snapshots	Qualità alimentare	assessment_id, predimed_score, adherence_band, ai_tips_json
activity_snapshots	Valutazione attività	assessment_id, minutes_per_week, sedentary_level, qualitative_band
followup_plans	Prossimi controlli	patient_id, next_review_date, timeline_json, owner_user_id
alerts	Alert clinici e operativi	id, patient_id, assessment_id, type, severity, status, due_at, audience
consent_records	Versionamento consensi	id, subject_type, subject_id, consent_type, legal_basis, policy_version, granted, granted_at, revoked_at
audit_events	Audit trail	id, tenant_id, actor_user_id, entity_type, entity_id, action, metadata_json, ip_hash, created_at
report_exports	Traccia export PDF/report	id, patient_id, assessment_id, exported_by, export_type, created_at
notification_jobs	Invio notifiche	id, patient_id, channel, payload_json, status, scheduled_at, sent_at

8. Regole dati e migrazione dal modello attuale

- **Migrazione chiave primaria logica:** aggiungere user_id uuid references auth.users(id) alle tabelle legacy e popolare retroattivamente dagli account esistenti; poi rimuovere dipendenza applicativa da email.
- **Anagrafica legacy:** trasformare anagrafica_utenti in fonte temporanea di import; successivamente splittare i dati tra patients, patient_clinical_profiles, assessments, assessment_measurements, nutrition_snapshots, activity_snapshots.
- **Snapshot temporali:** non sovrascrivere i parametri clinici ad ogni salvataggio; ogni visita genera un nuovo assessment con misure e score.

- **Consensi:** non memorizzare solo il consenso attuale; salvare evento di consenso con policy_version, timestamp, ip/device hash minimizzato e finalità.
- **Immagini:** spostare avatar/file in storage privato con signed URL temporanei, evitando blob/base64 nel record clinico.

9. Ruoli e modello autorizzativo

Ogni query clinica deve essere protetta da RLS per tenant e ruolo. Il frontend non deve mai poter interrogare direttamente dati di pazienti non assegnati. Le operazioni sensibili devono transitare per endpoint server-side con audit.

Ruolo	Permessi
platform_admin	gestione tenant, billing, audit globale, supporto limitato e tracciato
tenant_admin	gestione professionisti del tenant, impostazioni organizzazione, template PDF/logo
clinician	crea pazienti, visualizza pazienti propri o assegnati, crea assessment, exporta report, gestisce follow-up
assistant_staff	accesso limitato a dati amministrativi e scheduling, non a dati clinici completi se non necessario
patient	accesso al proprio portale, visualizzazione trend, reminder, consensi, report condivisi
service_role backend	solo lato server per job interni, mai esposto al client

10. RLS e sicurezza Supabase

9. Abilitare RLS su tutte le tabelle sanitarie e di tenant.
10. Creare policy di isolamento tenant robuste e verificabili.
11. Creare policy specifiche per clinician/patient: il paziente vede solo i propri dati; il professionista solo i pazienti assegnati o creati entro tenant.
12. Usare view o RPC server-side per query complesse invece di esporre join delicate dal client.
13. Eliminare duplicazione di project URL/anon key e centralizzare la configurazione.
14. Revocare eventuali policy permissive legacy prima della pubblicazione.
15. Separare completamente service role da frontend e CI client-side.

11. Vulnerabilità attuali da correggere

Problema	Rischio	Correzione richiesta
Supabase URL + anon key replicati in più file	configurazione dispersa, errore operativo, exposure governance	centralizzare config e ridurre superfici; nessun segreto nel client oltre chiave pubblica

Problema	Rischio	Correzione richiesta
		strettamente necessaria
Query .eq("email", email)	IDOR/logica fragile, mismatch email, impossibilità multi-paziente	migrare a user_id, patient_id, tenant_id
postMessage("*") tra iframe	message injection / confusione origine	validare origin e schema payload oppure eliminare iframe architecture
Listener globali multipli per risultati score	stati inconsistenti, doppio salvataggio	unificare orchestration nel clinical engine
Avatar in base64 nel record	payload enorme, leakage, performance	object storage privato + signed URL
Feature AI lato salute poco governate	rischio medico-legale e GDPR	LLM opzionale solo per sintesi non prescrittive e con consenso separato
Dashboard con training plan e nutrizione prescrittiva	verticale confuso	de-scope a monitoraggio qualità dieta e attività
Consenso solo UI	non sufficiente per accountability GDPR	registro eventi, versioning, revoke flow, proof
HTML statico con logica clinica e DB	manutenibilità scarsa	modularizzare frontend/back/domain
Mancanza evidente di audit e retention	compliance debole	audit_events, retention rules, deletion/export flows

12. Cybersecurity baseline obbligatoria

- **Secret management:** chiavi server solo in env sicure; rotazione chiavi; separazione dev/staging/prod.
- **Headers e browser hardening:** Content-Security-Policy, X-Frame-Options / frame-ancestors, Referrer-Policy, Permissions-Policy, X-Content-Type-Options, HSTS se su dominio finale.
- **CSRF/XSS:** se usi cookie/sessioni server-side, introdurre protezione CSRF; validazione e sanitizzazione input; niente HTML non trusted renderizzato senza escaping.
- **Rate limiting:** login, export report, notifiche, endpoint AI, generazione PDF.
- **Abuse prevention:** captcha o friction al login/registrazione se pubblico, lockout soft, audit accessi anomali.
- **Logging e monitoring:** error tracking, audit security events, alerting su spike di accessi/export, logging privacy-safe.
- **Backups e recovery:** backup periodici database e storage, recovery runbook, test ripristino.
- **Dependency hygiene:** pin versioni, scansione dipendenze, aggiornamenti controllati.

13. GDPR / privacy by design

Per dati sanitari il GDPR va trattato come requisito di architettura, non di interfaccia.

16. Definire ruoli privacy: titolare, responsabile, sub-responsabili, provider AI, hosting e notifiche.
17. Documentare finalità distinte: valutazione rischio, follow-up, export report, alert paziente, analytics tecnici anonimi.
18. Raccogliere consensi distinti dove necessario: trattamento dati salute, notifiche paziente, uso AI opzionale, marketing separato se mai esistesse.
19. Salvare policy_version, timestamp, attore e stato del consenso; rendere possibile la revoca.
20. Applicare minimizzazione: raccogli solo i dati necessari al verticale cardio-nefro-metabolico.
21. Implementare retention policy.
22. Fornire export dei dati e cancellazione/anonimizzazione dove compatibile con obblighi di conservazione.
23. Redigere DPIA se il modello operativo lo richiede.
24. Nessun dato sanitario verso provider AI senza base giuridica e controllo dei campi inviati.
25. Separare analytics di prodotto da dati sanitari identificativi.

14. Clinical engine: progettazione

Il clinical engine deve essere il cuore del sistema. Non deve risiedere in iframe HTML separati. Le formule attuali vanno riusate o portate in funzioni pure. La UI chiama un solo orchestratore che produce uno snapshot coerente.

Modulo	Responsabilità
score-engine	esegue le formule dei singoli score senza effetti collaterali
risk-aggregation	combina score e fattori in bucket integrato cardio-nefro-metabolico
screening-engine	genera screening e visite consigliate in base a età, sesso, rischio, score e regole
followup-engine	calcola tempistiche follow-up e prossime azioni
nutrition-engine	stima qualità dieta/aderenza e suggerimenti brevi non prescrittivi
activity-engine	interpreta attività fisica come esposizione / fattore di rischio
alert-engine	crea alert per peggioramento, mancanza dati, scadenze, red flags
report-engine	compone payload PDF clinico e patient summary

15. Score da mantenere e score aggiuntivi consigliati

- **Da mantenere sicuramente:** SCORE2, SCORE2-Diabetes, ADA Diabetes Risk, FLI, BMI, sindrome metabolica, parametri lipidi/glicemia/pressione, PREDIMED come qualità dieta.
- **FRAIL:** mantenerlo come modulo opzionale o secondario, più utile in coorti anziane; non core del messaggio commerciale principale.

- **Da NON mettere nel core se non già robusti:** FRAX o moduli fuori verticale principale, se complicano il posizionamento.
- **Aggiunta consigliata solo se dati disponibili:** CKD risk / albuminuria + eGFR staging, oppure almeno classificazione renale di base, perché il verticale richiesto è cardio-nefro-metabolico e oggi la componente nefro appare meno esplicita.
- **Regola:** qualunque nuovo score deve entrare solo se clinicamente coerente, deterministicamente implementabile e con dati disponibili.

16. Pseudocodice del clinical engine

```
function buildAssessmentSnapshot(input):
  validateInput(input)
  normalized = normalizeClinicalInput(input)

  scoreResults = {}
  scoreResults.score2 = computeScore2(normalized)
  if normalized.hasDiabetes:
      scoreResults.score2Diabetes = computeScore2Diabetes(normalized)
  scoreResults.ada = computeAda(normalized)
  scoreResults.fli = computeFli(normalized)
  scoreResults.frail = maybeComputeFrail(normalized)
  scoreResults.metabolicSyndrome = computeMetabolicSyndrome(normalized)
  scoreResults.bmi = computeBmi(normalized)
  scoreResults.predimed = computePredimed(normalized)
  scoreResults.ckd = maybeComputeKidneyRisk(normalized)

  compositeRisk = aggregateCardioNephroMetabolicRisk({
      demographics: normalized.demographics,
      vitals: normalized.vitals,
      labs: normalized.labs,
      scoreResults
  })

  screenings = determineRequiredScreenings({
      demographics: normalized.demographics,
      scoreResults,
      diagnoses: normalized.diagnoses
  })

  followupPlan = determineFollowupPlan({
      compositeRisk,
      scoreResults,
      missingDataFlags: normalized.missingDataFlags
  })

  nutritionSummary = buildNutritionSummary({
      predimed: scoreResults.predimed,
      lifestyle: normalized.lifestyle
  })

  activitySummary = buildActivitySummary({
      weeklyMinutes: normalized.activity.weeklyMinutes,
      sedentaryLevel: normalized.activity.sedentaryLevel
  })

  alerts = deriveAlerts({
      previousAssessment: input.previousAssessment,
```

```
    currentCompositeRisk: compositeRisk,
    scoreResults,
    followupPlan,
    missingDataFlags: normalized.missingDataFlags
})

reportPayload = buildClinicalReportPayload({
    patient: normalized.patient,
    assessmentDate: normalized.assessmentDate,
    scoreResults,
    compositeRisk,
    screenings,
    followupPlan,
    nutritionSummary,
    activitySummary,
    alerts
})

return {
    normalizedInput: normalized.redactedView,
    scoreResults,
    compositeRisk,
    screenings,
    followupPlan,
    nutritionSummary,
    activitySummary,
    alerts,
    reportPayload
}
```

17. Alert intelligenti

Gli alert devono essere utili, non rumorosi. Servono sia al professionista sia, se il professionista lo attiva, al paziente.

Tipo alert	Trigger	Audience	Azione
clinical_risk_up	peggioramento bucket di rischio o incremento score oltre soglia	clinician	review anticipata
followup_due	scadenza controllo / esami	clinician + patient	reminder
missing_critical_data	parametri mancanti per score core	clinician	richiedi completamento
red_flag	pattern ad alta priorità da regola deterministica	clinician	presa in carico rapida
diet_adherence_drop	calo PREDIMED/aderenza tra valutazioni	patient + clinician	micro-tip e review
activity_decline	minuti attività in forte calo o sedentarietà elevata	patient + clinician	promemoria non prescrittivo
consent_expired_or_re	consenso AI/notifiche	system/admin	blocca funzionalità

Tipo alert	Trigger	Audience	Azione
voked	revocato o versione policy obsoleta		corrispondente

18. Report PDF professionale

- **Contenuti:** identificativo paziente, dati essenziali visita, score principali, bucket integrato, interpretazione, screening/follow-up, trend, alert, note cliniche, disclaimer.
- **Branding:** logo e dati del tenant/clinica, nome del professionista, eventuale firma digitale/placeholder.
- **Formato:** struttura leggibile per medico e paziente, tabelle sintetiche, blocco trend, legenda colori controllata.
- **Sicurezza:** generazione solo server-side, URL temporanei o download autenticato, audit di ogni export.
- **Versioning:** report ricollegato ad assessment_id e versione algoritmo/report.

19. API schema minimo

Metodo	Endpoint	Scopo
POST	/api/auth/session	bootstrap sessione / refresh contesto utente
GET	/api/me	profilo utente autenticato e ruolo
GET	/api/patients	lista pazienti assegnati al professionista
POST	/api/patients	creazione paziente
GET	/api/patients/:id	dettaglio paziente con summary
PATCH	/api/patients/:id	aggiornamento dati anagrafici/stabili
GET	/api/patients/:id/assessments	storico valutazioni
POST	/api/patients/:id/assessments	crea assessment e calcola snapshot clinico
GET	/api/assessments/:id	dettaglio assessment
POST	/api/assessments/:id/report	genera o scarica report PDF
GET	/api/patients/:id/alerts	alert aperti e storici
POST	/api/alerts/:id/ack	presa visione / chiusura alert
POST	/api/consents	salva evento consenso
GET	/api/consents/:subjectId	storico consensi
POST	/api/notifications/test	invio controllato test notifiche
GET	/api/admin/audit	audit filtered per admin autorizzati

20. Contratti payload essenziali

```
POST /api/patients/:id/assessments
{
  "assessmentDate": "2026-04-19",
  "measurements": {
    "heightCm": 175,
    "weightKg": 89,
    "waistCm": 108,
    "sbp": 145,
    "dbp": 88,
    "totalCholMgDl": 220,
    "hdlMgDl": 42,
    "ldlMgDl": 140,
    "triglyceridesMgDl": 210,
    "glucoseMgDl": 112,
    "hba1cPct": 6.1,
    "egfr": 78,
    "ggt": 52,
    "ast": 28,
    "alt": 41,
    "platelets": 230
  },
  "lifestyle": {
    "smoking": true,
    "weeklyActivityMinutes": 60,
    "predimedAnswers": { "...": "..." }
  },
  "clinicalContext": {
    "hasDiabetes": true,
    "ageAtDiabetesDiagnosis": 49,
    "familyHistoryDiabetes": true,
    "hypertension": true,
    "medications": ["statin", "ace-inhibitor"]
  },
  "notes": "controllo trimestrale"
}

Response 201
{
  "assessmentId": "...",
  "scoreResults": { ... },
  "compositeRisk": { ... },
  "followupPlan": { ... },
  "alerts": [ ... ],
  "reportAvailable": true
}
```

21. Frontend UX target

- **Homepage B2B:** messaggio per professionisti, non per wellness consumer.
- **Dashboard professionista:** lista pazienti, alert aperti, follow-up in scadenza, nuovi report, trend rischio.
- **Scheda paziente:** overview clinica, timeline valutazioni, score principali, aderenza dieta, attività, note, export PDF.
- **Assessment form:** form modulare con sezioni parametri, storia clinica, alimentazione, attività, consensi e note.

- **Portale paziente:** vista semplificata con trend, reminder, qualità dieta, attività, consensi, report condivisi.
- **No clutter:** rimuovere schede irrilevanti, tab duplicate, iframe nascosti, pulsanti consumer e flussi chatbot.

22. Piano di refactor file-by-file

File attuale	Intervento richiesto
chatbot.html / chat.html / chatbot-logic.js	rimuovere dal flusso principale; se si mantiene AI, convertirla in modulo backend per sintesi/report e tips non prescrittivi, non chatbot generalista
dashboard.html	ridisegnare come dashboard B2B; eliminare piano allenamento e sezioni fuori verticale; introdurre overview pazienti/alert/report
dashboard-logic.js	smontare logica monolitica; spostare dominio clinico in moduli puri e dati su assessment snapshot
profilo.html	trasformare da profilo single-user a scheda paziente/edit anagrafica + fattori stabili; rimuovere blob/base64 avatar inline
score2*.js / ada-score.js / fli-score.js / frail-score.js	estrarre logica di caricamento dati e persistenza; mantenere formule ma eliminare dipendenza iframe/postMessage
score2.html / score2-diabetes.html / ADA-score.html / FLI.html / FRAIL.html	riutilizzare eventualmente il codice formula, ma non come pagine separate di runtime; convertirle in moduli o test fixtures
openai.js	ridurre scope; niente allenamento/piano alimentare; AI solo per tips alimentari leggeri, sintesi report e testo paziente sotto consenso separato
login.html	hardening auth UX, rate limit, messaging, legal links, versioning policy
index.html	ripositioning completo verso B2B clinico

23. Test strategy obbligatoria

26. Unit test per ogni formula score per garantire equivalenza pre/post refactor.
27. Golden tests su casi clinici noti e fixture legacy.
28. Integration test su creazione paziente, assessment, report PDF, alert e consensi.
29. Security test su RLS: utente A non può vedere paziente di utente B / altro tenant.
30. E2E test su login, dashboard clinician, scheda paziente, export report.
31. Regression test sul clinical engine con snapshot JSON.
32. Performance test minimo su rendering dashboard, report e query storiche.

24. Modello SaaS medico B2B

- **Cliente pagante:** studio medico, nutrizionista clinico, diabetologo, cardiologo, centro prevenzione, poliambulatorio.
- **Unità di valore:** numero professionisti, numero pazienti attivi, export report, alert, branding, multi-tenant.
- **Piani suggeriti:** Starter, Professional, Clinic, Enterprise/White-label.
- **Driver di vendita:** report professionale, tempo risparmiato in visita, follow-up strutturato, storico, alert, aderenza paziente, verticalità clinica.
- **Feature premium:** white-label, template PDF personalizzati, reminder SMS/email, analytics clinica di coorte, integrazione cartella clinica, ruoli staff.
- **Metriche prodotto:** pazienti attivi per professionista, assessment/mese, tasso follow-up, aderenza dieta, alert chiusi, report esportati.

25. Sequenza di implementazione raccomandata

Fase	Obiettivo	Output
1	Hardening sicurezza + migrazione identità	user_id, tenant model, RLS, config centralizzata
2	De-scope prodotto	rimozione chatbot generalista, training plan, piano alimentare
3	Clinical engine unificato	moduli puri, test equivalenza score, snapshot assessment
4	Schema B2B multi-paziente	patients, assessments, score_results, alerts, consents
5	Nuova UX professionista	dashboard pazienti, scheda paziente, storico
6	Report PDF e alert	report server-side, notifiche, audit export
7	GDPR operationalization	consent versioning, retention, export/deletion workflow
8	Beta pubblica controllata	monitoring, incident playbook, billing/pricing pages

26. Definition of done

- **Tecnico:** nessuna query clinica basata su email, RLS attiva e testata, clinical engine centralizzato, API documentate, report server-side.
- **Prodotto:** solo verticale cardio-nefro-metabolico, nutrizione come monitoraggio, attività fisica come fattore di rischio, multi-paziente attivo.
- **Sicurezza:** CSP e header attivi, secret management corretto, audit, rate limit, test di isolamento tenant.

- **GDPR:** consensi versionati, privacy links e documenti, minimizzazione dati, logging privacy-safe, export/cancellazione governati.
- **Business:** dashboard B2B credibile, report vendibile, alert intelligenti, posizionamento chiaro.

27. Prompt esecutivo finale per Claude Code / Claude CoWork

Copia-incolla il seguente prompt in Claude Code / Claude CoWork come task principale. È scritto per guidarlo come senior engineer/architect.

```
Agisci come Principal Engineer, Security Engineer, Staff Product Architect e Health SaaS
CTO.
Devi rifattorizzare e completare questa web app esistente trasformandola in una piattaforma
B2B per la valutazione e il monitoraggio del rischio cardio-nefro-metabolico.

CONTESTO E OBIETTIVO
- L'app attuale contiene score clinici, dashboard, profilo, chatbot, nutrizione e attività
fisica.
- Il target finale NON è una wellness app consumer.
- Il target finale è una piattaforma B2B per professionisti sanitari con multi-paziente,
storico valutazioni, report PDF, alert intelligenti e portale paziente controllato.
- Devi mantenere le logiche di calcolo degli score già esistenti e testate. NON devi
modificare formule, soglie o calcoli senza una prova necessaria e test di equivalenza.
- Devi eliminare la parte di programma di allenamento e la parte di piano alimentare
completo.
- Devi mantenere: 1) valutazione qualità alimentare / aderenza dieta mediterranea 2)
valutazione attività fisica solo come fattore di rischio e monitoraggio 3) score clinici
core 4) report PDF 5) storico pazienti 6) alert intelligenti per professionista e paziente

VINCOLI DI IMPLEMENTAZIONE
- Non usare email come chiave logica applicativa.
- Migra tutto a user_id, tenant_id, patient_id, assessment_id.
- Abilita e testa RLS su tutte le tabelle sensibili.
- Elimina l'architettura basata su iframe e postMessage per gli score, salvo usarla come
riferimento temporaneo per estrarre le formule.
- Centralizza il clinical engine in moduli puri.
- Sposta le operazioni sensibili su backend/API server-side.
- Non salvare avatar o file in base64 nei record clinici.
- Implementa object storage privato con signed URL per eventuali immagini/documenti.
- Mantieni AI solo per: a) tips alimentari leggeri e non prescrittivi b) sintesi testuale di
report c) messaggi paziente semplici, sempre dietro consenso separato e senza inviare dati
non necessari.
- Non reintrodurre chatbot generalista, symptom checker, training program o piano alimentare
prescrittivo.

FAI PRIMA UN AUDIT DELLA CODEBASE
1. Mappa i file, i flussi e le dipendenze attuali.
2. Identifica tutte le query basate su email.
3. Identifica tutte le chiavi/configurazioni esposte.
4. Identifica tutte le logiche duplicate e gli score isolati in pagine/iframe.
5. Elenca in modo esplicito le modifiche breaking e non-breaking.

POI IMPLEMENTA IL TARGET ARCHITECTURE
- Frontend modulare
- Backend/API per report, alert, consensi, audit
- Clinical engine: /domain/clinical/score-engine /risk-aggregation /screening-engine
/followup-engine /nutrition-engine /activity-engine /alert-engine
- Shared schemas e types
- Tests unit/integration/e2e
```

SCHEMA DATABASE TARGET
Crea o migra verso queste entità: tenants, users, professionals, patients, patient_clinical_profiles, assessments, assessment_measurements, score_results, risk_profiles, nutrition_snapshots, activity_snapshots, followup_plans, alerts, consent_records, audit_events, report_exports, notification_jobs.

SICUREZZA E COMPLIANCE
- Implementa RLS robusta per tenant e ruolo.
- Nessun accesso cross-tenant.
- Aggiungi audit trail per accessi/modifiche/export.
- Centralizza config pubblica.
- Secret management corretto.
- Implementa CSP, Referrer-Policy, Permissions-Policy, X-Content-Type-Options, frame-ancestors.
- Se usi cookie/sessioni server, aggiungi CSRF protection dove applicabile.
- Rate limit per login, export, report, notifiche, AI.
- Logging privacy-safe.
- Implementa versioning dei consensi con policy_version, granted_at, revoked_at, legal_basis.
- Prepara retention policy e workflow export/cancellazione.

CLINICAL ENGINE
- Mantieni le formule esistenti di SCORE2, SCORE2-Diabetes, ADA, FLI, FRAIL, BMI, sindrome metabolica, PREDIMED.
- Crea test di equivalenza pre/post refactor per i singoli score.
- Costruisci una funzione orchestratrice che da un assessment produce: score_results, composite cardio-nephro-metabolic risk, screenings, followup_plan, nutrition_summary, activity_summary, alerts, report_payload.
- Se reputi clinicamente indispensabile per il verticale cardio-nefro-metabolico, proponi l’aggiunta di una classificazione nefrologica di base con eGFR e/o albuminuria, ma solo se implementabile deterministicamente e con dati disponibili.

UX/PRODOTTO
- Trasforma dashboard.html in una dashboard B2B per professionista.
- Crea lista pazienti.
- Crea scheda paziente con storico e trend.
- Crea assessment form e timeline.
- Crea report PDF professionale server-side.
- Crea portale paziente semplificato con alert/reminder/trend.
- Mantenere nutrizione solo come monitoraggio qualità dieta e micro-tips.
- Mantenere attività fisica solo come valutazione clinica / esposizione, non programma di allenamento.

OUTPUT RICHIESTI
Voglio che tu produca in sequenza: 1. audit tecnico dettagliato della codebase 2. piano di refactor 3. schema database SQL/migrazioni 4. architettura cartelle finale 5. clinical engine modules 6. refactor dei file esistenti 7. API routes 8. report PDF implementation 9. test suite 10. checklist sicurezza e GDPR 11. changelog finale

MODO DI LAVORARE
- Procedi per step piccoli e verificabili.
- Mostra sempre diff e motivazione delle modifiche.
- Non introdurre regressioni sui calcoli degli score.
- Quando trovi logiche legacy buone, riusale; quando trovi UI/architettura sbagliata, sostituiscila.
- Se una decisione è dubbia, scegli l’opzione più sicura, più semplice da manutenere e più coerente con il verticale B2B.

28. Checklist finale di esecuzione

33. Salvare questo documento come specifica di progetto.
34. Usarlo come prompt master in Claude Code / CoWork.
35. Far partire Claude dall’audit e non direttamente dalla riscrittura cieca.
36. Validare subito la migrazione user_id / tenant / patient / assessment.
37. Pretendere test equivalenza sugli score prima di qualsiasi rilascio.
38. Bloccare il go-live finché non sono completati hardening e RLS.